

PSET3

❧ Problema 0: text_to_list

Hay que transformar un string (texto completo de un archivo) en una lista de palabras.

Ejemplo: 'hello world hello' → ['hello', 'world', 'hello']

Ya hay una función llamada `load_file` que limpia el texto y te da el string. Vos te encargás de dividirlo en palabras.

```
1  ### Problem 0: Prep Data ###
2  def text_to_list(input_text):
3      """
4      Args:
5          input_text: string representation of text from file.
6                      assume the string is made of lowercase characters
7      Returns:
8          list representation of input_text, where each word is a different
9          element in the list
10     """
11     return input_text.split()
```

1234 Problema 1: get_frequencies

Dado una lista de palabras (o letras), devolvés un diccionario donde las claves son los elementos y los valores son cuántas veces aparecen.

Ejemplo: ['hello', 'world', 'hello'] → {'hello': 2, 'world': 1}

Es una función típica de conteo de frecuencia, muy usada en NLP.

```
1  ### Problem 1: Get Frequency ###
2  def get_frequencies(input_iterable):
3      """
4      Args:
5          input_iterable: a string or a list of strings, all are made of
6                          lowercase characters
7      Returns:
8          dictionary that maps string:int where each string
9          is a letter or word in input_iterable and the corresponding int
10         is the frequency of the letter or word in input_iterable
11     Note:
12         You can assume that the only kinds of white space in the text
13         documents we provide will be new lines or space(s) between words (i.e.
```

```

    there are no tabs)
12     """
13     dictionary = {}
14     for word in input_iterable:
15         if word in dictionary:
16             dictionary[word] += 1
17         else:
18             dictionary[word] = 1
19     return dictionary

```

Problema 2: `get_letter_frequencies`

Dado un string (una palabra), devolvés cuántas veces aparece cada letra.

Ejemplo: `'hello'` → `{'h': 1, 'e': 1, 'l': 2, 'o': 1}`

Debes usar la función del problema anterior (`get_frequencies`) para implementarla, lo cual te obliga a pensar en reutilización de código.

```

1  ### Problem 2: Letter Frequencies ###
2  def get_letter_frequencies(word):
3      """
4      Args:
5          word: word as a string
6      Returns:
7          dictionary that maps string:int where each string
8          is a letter in word and the corresponding int
9          is the frequency of the letter in word
10     """
11     return get_frequencies(word)

```

Problema 3: `calculate_similarity_score`

Acá se pone interesante. Te dan dos diccionarios (de frecuencias) y debes calcular qué tan similares son.

La fórmula se basa en la suma de las diferencias absolutas entre las frecuencias, normalizadas por la suma total. El resultado va de 0 (muy distintos) a 1 (idénticos).

Usa letras o palabras, según el contexto.

```

1  ### Problem 3: Similarity ###
2  def calculate_similarity_score(freq_dict1, freq_dict2):
3      """
4      The keys of dict1 and dict2 are all lowercase,
5      you will NOT need to worry about case sensitivity.

```

```

6
7     Args:
8         freq_dict1: frequency dictionary of letters of word1 or words of
text1
9         freq_dict2: frequency dictionary of letters of word2 or words of
text2
10    Returns:
11        float, a number between 0 and 1, inclusive
12        representing how similar the words/texts are to each other
13
14    The difference in words/text frequencies = DIFF sums words
15    from these three scenarios:
16    * If an element occurs in dict1 and dict2 then
17      get the difference in frequencies
18    * If an element occurs only in dict1 then take the
19      frequency from dict1
20    * If an element occurs only in dict2 then take the
21      frequency from dict2
22    The total frequencies = ALL is calculated by summing
23    all frequencies in both dict1 and dict2.
24    Return 1-(DIFF/ALL) rounded to 2 decimal places
25    """
26    all_keys = set(freq_dict1) | set(freq_dict2)
27    diff = 0
28    total = 0
29
30    for key in all_keys:
31        f1 = freq_dict1.get(key, 0)
32        f2 = freq_dict2.get(key, 0)
33        diff += abs(f1 - f2)
34        total += f1 + f2
35
36    if total == 0:
37        return 1.0
38
39    return round(1 - (diff / total), 2)

```

Problema 4: `get_most_frequent_words`

Te dan dos diccionarios de frecuencia (de palabras). Debes combinar los valores por palabra (sumarlos) y devolver la(s) palabra(s) más frecuentes. Si hay empate, devolvés una lista ordenada alfabéticamente.

Ejemplo: `{"hello": 5, "world": 1}` y `{"hello": 1, "world": 5}` → `["hello", "world"]`

```

1    ### Problem 4: Most Frequent Word(s) ###
2    def get_most_frequent_words(freq_dict1, freq_dict2):
3        """
4        The keys of dict1 and dict2 are all lowercase,
5        you will NOT need to worry about case sensitivity.

```

```

6
7     Args:
8         freq_dict1: frequency dictionary for one text
9         freq_dict2: frequency dictionary for another text
10    Returns:
11        list of the most frequent word(s) in the input dictionaries
12
13    The most frequent word:
14        * is based on the combined word frequencies across both
15    dictionaries.
16        If a word occurs in both dictionaries, consider the sum the
17        frequencies as the combined word frequency.
18        * need not be in both dictionaries, i.e it can be exclusively in
19        dict1, dict2, or shared by dict1 and dict2.
20    If multiple words are tied (i.e. share the same highest frequency),
21    return an alphabetically ordered list of all these words.
22    """
23    all_words = set(freq_dict1) | set(freq_dict2)
24
25    combined_freq = {}
26
27    for word in all_words:
28        combined_freq[word] = freq_dict1.get(word, 0) +
29        freq_dict2.get(word, 0)
30
31    val_max = max(combined_freq.values())
32
33    return sorted([w for w in combined_freq if combined_freq[w] ==
34    val_max])

```

Problema 5: TF-IDF

Entramos al terreno del análisis de texto un poco más avanzado. Hay tres funciones:

a) `get_tf`

Dado un archivo, devolvés un diccionario con cada palabra y su frecuencia relativa (frecuencia de esa palabra ÷ total de palabras). Ejemplo: si hay 3 palabras y 2 son "hello", el TF de "hello" es 2/3.

```

1    ### Problem 5: Finding TF-IDF ###
2    def get_tf(file_path):
3        """
4        Args:
5            file_path: name of file in the form of a string
6        Returns:
7            a dictionary mapping each word to its TF
8
9            * TF is calculatd as TF(i) = (number times word *i* appears
10            in the document) / (total number of words in the document)

```

```

11     * Think about how we can use get_frequencies from earlier
12     """
13     file = load_file(file_path)
14     text = text_to_list(file)
15     freq = get_frequencies(text)
16     dictionary = {}
17     for word in freq:
18         dictionary[word] = freq[word] / len(text)
19     return dictionary

```

b) get_idf

Dado varios archivos, devolvés para cada palabra su IDF, una medida que dice cuán rara es esa palabra en el conjunto. Si aparece en todos, $IDF = 0$. Si en pocos, $IDF > 0$.

```

1  def get_idf(file_paths):
2      """
3      Args:
4          file_paths: list of names of files, where each file name is a
                        string
5      Returns:
6          a dictionary mapping each word to its IDF
7
8      * IDF is calculated as  $IDF(i) = \log_{10}(\text{total number of documents} /$ 
        number of
9      documents with word *i* in it), where  $\log_{10}$  is log base 10 and can be
        called
10     with math.log10()
11
12     """
13     word_doc_count = {}
14     total_docs = len(file_paths)
15
16     for path in file_paths:
17         file = load_file(path)
18         words = set(get_frequencies(text_to_list(file)).keys())
19
20         for word in words:
21             word_doc_count[word] = word_doc_count.get(word, 0) + 1
22
23     idf = {}
24     for word, doc_count in word_doc_count.items():
25         if doc_count == total_docs:
26             idf[word] = 0.0
27         else:
28             idf[word] = math.log10(total_docs / doc_count)
29
30     return idf

```

c) get_tfidf

Usas los resultados anteriores: para un archivo y un conjunto de archivos, devolvés la TF-IDF de cada palabra. Lo ordenas por valor creciente, y en caso de empate, por orden alfabético.

```
1  def get_tfidf(tf_file_path, idf_file_paths):
2      """
3          Args:
4              tf_file_path: name of file in the form of a string (used to
5              calculate TF)
6              idf_file_paths: list of names of files, where each file name
7              is a string
8              (used to calculate IDF)
9          Returns:
10             a sorted list of tuples (in increasing TF-IDF score), where
11             each tuple is
12             of the form (word, TF-IDF). In case of words with the same TF-
13             IDF, the
14             words should be sorted in increasing alphabetical order.
15
16             *  $TF-IDF(i) = TF(i) * IDF(i)$ 
17             """
18     tf = get_tf(tf_file_path)
19     idf = get_idf(idf_file_paths)
20     word_tfidf = []
21     for word in tf:
22         tfidf_score = tf[word] * idf.get(word, 0)
23         word_tfidf.append((word, tfidf_score))
24     return sorted(word_tfidf, key=lambda x: (x[1], x[0]))
```